PART 2: Core Syntax Deep-Dives

2.1 String Architecture — The String Constant Pool

Definition & Internal Mechanics

`String` in Java is a `final` class backed by a `char[]` array (or `byte[]` in Java 9+ with compact strings for Latin-1). All `String` instances are **immutable** — once created, their character sequence never changes.

The String Constant Pool (SCP):

- Located in the **Heap** since Java 7+ (was in PermGen before Java 7)
- A cache of `String` instances to avoid redundant allocations
- Populated by **string literals** at class-load time
- `String.intern()` can manually add heap strings to the pool

Memory behavior:

- `String s = "hello"` → JVM checks the SCP; if `"hello"` exists, returns the same reference; otherwise creates a new object in the pool
- `String s = new String("hello")` → **Always** creates a new heap object (outside the pool), even if the pool has "hello"
- `"a" + "b"` → **Compile-time constant folding**: the compiler evaluates this to `"ab"` and uses the pool
- `s1 + s2` where s1/s2 are variables → Creates a new `String` on the heap via `StringBuilder` internally (not pooled)

Why Strings are immutable:

1. **Thread safety:** Immutable objects can be freely shared across threads without synchronization
2. **Security:** Strings are used for class names, file paths, network URLs — mutation would be a security risk
3. **HashCode caching:** `String` caches its `hashCode` (lazy, computed once, stored in a field) — safe only because of immutability
4. **String pool efficiency:** Sharing requires immutability — if strings were mutable, sharing would mean one thread could change another's string

Real-Life Analogy

The String Constant Pool is like a **government-issued social security number system** — each unique number (string value) is issued once and shared. Everyone who has "SSN 123-45-6789" shares the same entry in the registry. If you try to get a new SSN for an existing person, you get back the existing one. `new String("hello")` is like printing a photocopy of an existing ID — it's a new physical document (heap object) with the same information.

Interview Context & Why It Matters

This topic reveals whether a candidate understands memory architecture, reference equality vs value equality, and the performance impact of string concatenation in loops (a very common interview question).

Expression	Pool?	Notes
`"hello"`	✅ Yes	Literal goes to pool
`new String("hello")`	❌ No	Forced heap allocation
`"hello".intern()`	✅ Yes	Manually adds to pool
`s1 + s2` (vars)	❌ No	Heap via StringBuilder
`"a" + "b"` (literals)	✅ Yes	Compile-time fold → "ab"

Production-Ready Code Snippet

java

```java
/**
 * Demonstrates String Constant Pool behavior, reference vs. value equality,
 * intern(), and the StringBuilder performance optimization for loops.
 */
public class StringArchitectureDemo {

    public static void main(String[] args) {
        // ---- String Constant Pool ----
        String s1 = "hello";
        String s2 = "hello";        // Same pool reference as s1
        String s3 = new String("hello"); // New heap object — NOT from pool
        String s4 = s3.intern();    // s4 now points to the pool entry

        System.out.println(s1 == s2);   // true  — same pool reference
        System.out.println(s1 == s3);   // false — s3 is on heap
        System.out.println(s1 == s4);   // true  — s4 interned back to pool
        System.out.println(s1.equals(s3)); // true — value equality always correct

        // ---- Compile-time constant folding ----
        String compileTime = "hel" + "lo"; // Folded at compile time → pool entry
        System.out.println(s1 == compileTime); // true — same pool entry

        final String finalPart1 = "hel";
        final String finalPart2 = "lo";
        String finalConcat = finalPart1 + finalPart2; // final vars: also compile-ti
        System.out.println(s1 == finalConcat); // true

        String nonFinal1 = "hel";  // not final
        String nonFinal2 = "lo";
        String runtimeConcat = nonFinal1 + nonFinal2; // Runtime: new heap String
        System.out.println(s1 == runtimeConcat); // false

        // ---- Performance: String vs StringBuilder in loops ----
        int iterations = 10_000;

        // BAD: O(n^2) — each + creates a new String object and copies all previous
        long start = System.nanoTime();
        String bad = "";
        for (int i = 0; i < iterations; i++) {
            bad += "x"; // Creates 10,000 intermediate String objects!
        }
        long badTime = System.nanoTime() - start;
```

```
        // GOOD: O(n) — StringBuilder uses a resizable char array
        start = System.nanoTime();
        StringBuilder sb = new StringBuilder(iterations); // pre-size capacity
        for (int i = 0; i < iterations; i++) {
            sb.append("x");
        }
        String good = sb.toString();
        long goodTime = System.nanoTime() - start;

        System.out.printf("String concat: %,d ns%n", badTime);
        System.out.printf("StringBuilder: %,d ns%n", goodTime);
        System.out.println("Same result: " + bad.equals(good)); // true

        // ---- StringBuffer (thread-safe but slower than StringBuilder) ----
        StringBuffer threadSafeBuffer = new StringBuffer();
        // All methods are synchronized — use only when multiple threads access the
        // In practice, prefer StringBuilder + thread-per-string pattern instead
        threadSafeBuffer.append("thread-safe");
        System.out.println(threadSafeBuffer.toString());
    }
}
```

Common Interview Question & Expert Answer

Q: "What is the output of `System.out.println("a" + "b" == "ab")` and why?"

A: "`true`. The expression `"a" + "b"` involves only string literals, so the Java compiler performs **compile-time constant folding** — it evaluates the concatenation at compile time and replaces it with the literal `"ab"`. Both `"a" + "b"` and `"ab"` therefore reference the same entry in the String Constant Pool, making `==` return `true`. If instead you wrote `String a = "a"; String b = "b"; System.out.println(a + b == "ab")`, the result would be `false` because `a` and `b` are non-final variables, so the concatenation is resolved at **runtime** using a `StringBuilder`, producing a new heap object not in the pool."

2.2 Java Generics — Type Erasure and Bounded Wildcards

Definition & Internal Mechanics

Generics provide compile-time type safety for parameterized types. At runtime, due to **type erasure**, generic type parameters are replaced with their upper bounds (or `Object` if unbounded).

Type Erasure:

- `List<String>` and `List<Integer>` are both `List` (raw type) at runtime
- Generic type information exists only in class files as metadata (via `Signature` attributes) but is NOT available via `instanceof` or reflective `getClass()` at runtime
- **Bridge methods:** When a generic type is erased, the compiler may generate synthetic bridge methods to maintain polymorphism. For example, if `class Foo<T> { T get() {...} }` is compiled, and a subclass overrides `get()` with a concrete type, a bridge method with the erased signature is generated to satisfy the vtable requirement

Bounded Wildcards — PECS Rule:

- **Producer Extends:** If a structure **produces** (provides) elements of type `T`, use `<? extends T>` — you can read `T` from it, but cannot write
- **Consumer Super:** If a structure **consumes** (accepts) elements of type `T`, use `<? super T>` — you can write `T` into it, but can only read `Object` from it

Why you can't add to `List<? extends T>`: The compiler doesn't know the exact subtype. If you have `List<? extends Number>`, the list could be a `List<Integer>`, `List<Double>`, etc. Adding a `Double` to what might be a `List<Integer>` would corrupt type safety.

Real-Life Analogy

Think of PECS in terms of a food delivery system:

- **Producer Extends** = a restaurant supply truck (producer of food). You can *take* any food off the truck, but you shouldn't add random items to the truck because you don't know which restaurant's specific supplies are already in there.
- **Consumer Super** = a dumpster (consumer of waste). You can *put* anything in it (food waste, paper, anything), but when you pull things out, you just get "garbage" (Object) — you don't know what specific type it was.

Interview Context & Why It Matters

Generics questions filter candidates who truly understand type safety at compile vs runtime. The PECS rule is a senior-level concept — most developers use raw types or unbounded wildcards incorrectly, causing `ClassCastException` at runtime that generics were supposed to prevent.

Production-Ready Code Snippet

```java
```

```java
import java.util.ArrayList;
import java.util.List;

/**
 * Demonstrates Java generics: bounded type parameters, PECS wildcards,
 * type erasure consequences, and bridge methods via generic override.
 */
public class GenericsDemo {

    // Generic class with bounded type parameter
    static class NumberBox<T extends Number & Comparable<T>> {
        private T value;

        public NumberBox(T value) {
            if (value == null) throw new IllegalArgumentException("Value cannot be n
            this.value = value;
        }

        public T get() { return value; }

        public boolean isGreaterThan(NumberBox<T> other) {
            return this.value.compareTo(other.value) > 0;
        }

        @Override
        public String toString() {
            return "NumberBox[" + value + "]";
        }
    }

    // Generic method
    static <T extends Comparable<T>> T max(T a, T b) {
        return a.compareTo(b) >= 0 ? a : b;
    }

    // PECS — Producer: read from source, write into destination
    // source produces elements (? extends T) — we READ from it
    // destination consumes elements (? super T) — we WRITE into it
    static <T> void copyElements(List<? extends T> source, List<? super T> destinati
        for (T element : source) {
            destination.add(element); // adding T to "? super T" is safe
        }
    }
```

```java
// Producer-only: can read Numbers from any List of Number subtype
static double sumList(List<? extends Number> numbers) {
    return numbers.stream()
                  .mapToDouble(Number::doubleValue)
                  .sum();
}

// Consumer-only: can add Integers to any List that accepts Integers or supertyp
static void addIntegers(List<? super Integer> list) {
    list.add(1);
    list.add(2);
    list.add(3);
    // Object x = list.get(0); // Only Object can be read back — type is unknown
}

public static void main(String[] args) {
    // Bounded generic class
    NumberBox<Integer> intBox = new NumberBox<>(42);
    NumberBox<Integer> otherBox = new NumberBox<>(10);
    System.out.println(intBox.isGreaterThan(otherBox)); // true

    // Generic method
    System.out.println(max("banana", "apple")); // banana (lexicographic)
    System.out.println(max(3.14, 2.71));         // 3.14

    // PECS demonstration
    List<Integer> integers = List.of(1, 2, 3, 4, 5);
    List<Double> doubles = List.of(1.5, 2.5, 3.5);

    System.out.println("Sum of integers: " + sumList(integers)); // 15.0
    System.out.println("Sum of doubles: " + sumList(doubles));   // 7.5

    List<Number> destination = new ArrayList<>();
    addIntegers(destination); // List<Number> is "? super Integer" — valid
    System.out.println("Added to Number list: " + destination);

    List<Object> objectList = new ArrayList<>();
    addIntegers(objectList); // List<Object> is also "? super Integer" — valid

    // copyElements: copy List<Integer> to List<Number>
    List<Integer> src = List.of(10, 20, 30);
    List<Number> dest = new ArrayList<>();
    copyElements(src, dest);
```

```
        System.out.println("Copied elements: " + dest);

        // Type erasure demonstration
        List<String> strings = new ArrayList<>();
        List<Integer> ints = new ArrayList<>();
        System.out.println("Same class at runtime: " + (strings.getClass() == ints.g
        System.out.println("Runtime class: " + strings.getClass().getName()); // jav
    }
}
```

Common Interview Question & Expert Answer

Q: "Why can't you do `new T[]` or `new ArrayList<T>()` inside a generic class where T is the type parameter? And what is a bridge method?"

A: "Both restrictions exist because of **type erasure**. At runtime, `T` is erased to its upper bound (usually `Object`), so the JVM has no idea what `T` is when it's time to create the array or collection. `new T[]` would be equivalent to `new Object[]` at runtime, but Java arrays carry their component type at runtime for covariance checking — this creates a type safety hole. Instead, you must either accept a `Class<T>` token and use `Array.newInstance(clazz, size)`, or accept the array from outside. Similarly, `new ArrayList<T>()` would create a `new ArrayList<Object>()` at runtime — you can do this, just with an unchecked cast. A **bridge method** is a synthetic method generated by the compiler to maintain polymorphism through type erasure. For example, if an interface declares `T getValue()` and is erased to `Object getValue()`, and your implementation declares `String getValue()`, the compiler generates a bridge method `Object getValue() { return (String) this.getValue(); }` — the bridge dispatches through vtable resolution to your actual implementation. You can see bridge methods with `Method.isBridge()` in reflection."

2.3 Java Records (Java 14+)

Definition & Internal Mechanics

Records are a special kind of class (immutable data carrier) introduced as a preview in Java 14, finalized in Java 16. They automatically generate:

- `private final` fields for each component
- Public accessor methods (not prefixed with `get`) — e.g., `name()` not `getName()`
- A canonical constructor
- `equals()` and `hashCode()` based on all components
- `toString()` including all component values

Records are implicitly `final` (cannot be subclassed) and their fields are **implicitly** `private final`.

Customization options:

- **Compact constructor:** Validate without re-assigning parameters (the compiler assigns them after your body)
- **Custom canonical constructor:** Full constructor with explicit assignment
- **Additional constructors:** Must delegate to the canonical constructor
- **Additional methods:** Allowed; no instance fields (only components are allowed)
- **Interface implementation:** Records can implement interfaces

Limitations:

- Cannot extend any class (implicitly extends `java.lang.Record`)
- No mutable state (all components are final)
- Cannot declare instance fields outside of components

Real-Life Analogy

A Java Record is like a **government-issued certificate** (birth certificate, diploma) — it's a standardized form. The data fields are fixed and printed, you can't modify them after issue, and anyone examining two certificates with the same data would consider them equivalent. The certificate "writes itself" based on what information you provide at creation.

Interview Context & Why It Matters

Records demonstrate knowledge of modern Java and the evolution toward less boilerplate. Interviewers test whether you know the compact constructor, serialization behavior, and when NOT to use records (when you need mutability, custom field names, or inheritance).

Production-Ready Code Snippet

java

```java
import java.util.List;
import java.util.Objects;

/**
 * Demonstrates Java Records: basic usage, compact constructors,
 * custom methods, interface implementation, and serialization behavior.
 */
public class RecordsDemo {

    // Basic Record — compiler generates all boilerplate
    record Point(double x, double y) {
        // Additional method on a record
        public double distanceTo(Point other) {
            Objects.requireNonNull(other, "Target point cannot be null");
            double dx = this.x - other.x;
            double dy = this.y - other.y;
            return Math.sqrt(dx * dx + dy * dy);
        }

        // Custom static factory
        public static Point origin() {
            return new Point(0, 0);
        }
    }

    // Record with compact constructor (validation)
    record Range(int min, int max) {
        // Compact constructor: no parameter list, compiler assigns fields after bod
        Range {
            if (min > max) {
                throw new IllegalArgumentException(
                    "Range invalid: min (%d) > max (%d)".formatted(min, max)
                );
            }
        }

        // Additional computed property (not a field — just a method)
        public int size() {
            return max - min;
        }

        public boolean contains(int value) {
            return value >= min && value <= max;
```

```
    }
}

// Record implementing an interface
interface Describable {
    String describe();
}

record Employee(String id, String name, String department) implements Describabl
    // Compact constructor with normalization
    Employee {
        Objects.requireNonNull(id, "ID required");
        Objects.requireNonNull(name, "Name required");
        name = name.trim(); // normalize: compact constructor can modify before
        department = department != null ? department.toUpperCase() : "UNASSIGNED
    }

    @Override
    public String describe() {
        return name + " (" + id + ") — " + department;
    }
}

// Nested records work well for hierarchical data
record Address(String street, String city, String country) {}
record Person(String name, int age, Address address) {
    Person {
        if (age < 0 || age > 150) throw new IllegalArgumentException("Invalid ag
        Objects.requireNonNull(name, "Name required");
        Objects.requireNonNull(address, "Address required");
    }
}

public static void main(String[] args) {
    // Point record usage
    Point p1 = new Point(3, 4);
    Point p2 = Point.origin();
    System.out.println(p1);                 // Point[x=3.0, y=4.0] — generated
    System.out.println(p1.x());             // 3.0 — accessor (not getX()!)
    System.out.println(p1.distanceTo(p2));  // 5.0

    // equals/hashCode generated from all components
    Point p3 = new Point(3, 4);
    System.out.println(p1.equals(p3));      // true
```

```
        System.out.println(p1 == p3);            // false — different objects

        // Range with compact constructor validation
        Range validRange = new Range(1, 10);
        System.out.println(validRange.size());        // 9
        System.out.println(validRange.contains(5));   // true
        try {
            Range invalid = new Range(10, 1);         // Throws IllegalArgumentExcep
        } catch (IllegalArgumentException e) {
            System.out.println("Caught: " + e.getMessage());
        }

        // Employee with normalization
        Employee emp = new Employee("E001", "  Alice  ", "engineering");
        System.out.println(emp.name());       // "Alice" — trimmed
        System.out.println(emp.department()); // "ENGINEERING" — uppercased
        System.out.println(emp.describe());   // Alice (E001) — ENGINEERING

        // Hierarchical records
        Person person = new Person("Bob", 30, new Address("123 Main St", "NYC", "USA
        System.out.println(person);

        // Records work well with pattern matching (Java 21)
        Object obj = new Point(1.5, 2.5);
        if (obj instanceof Point(double x, double y)) { // Deconstruction pattern
            System.out.println("Deconstructed: x=" + x + ", y=" + y);
        }
    }
}
```

Common Interview Question & Expert Answer

Q: "Can a Record in Java be mutable? Can you make a copy of a Record with one field changed?"

A: "Records are designed to be **immutable** — all components are `private final`, and there are no setters. You cannot change a component after construction. However, the immutability is **shallow** — if a component is a mutable object like `List`, you can mutate that list through its own reference (just like `final` fields in regular classes). For a true immutable record, use only immutable component types or use `Collections.unmodifiableList()` in the compact constructor. To create a modified copy, Java Records don't have a built-in `with()` method (unlike Kotlin data classes), but you can manually create a new record: `new Employee(original.id(), 'NewName', original.department())`. A common pattern is adding

builder-style `withName()` methods on the record itself that return a new instance. The JDK team has discussed adding `with` expressions in future versions under Project Amber."

2.4 Advanced Enums

Definition & Internal Mechanics

Java `enum` is a special class type where all instances are compile-time constants. Under the hood:

- Each enum constant is a `public static final` instance of the enum class
- The enum class implicitly extends `java.lang.Enum<E>` (cannot extend other classes)
- Enums can implement interfaces
- Enums can have fields, constructors (always `private`), and methods
- **Enum with abstract method:** Each constant provides its own implementation (anonymous inner class body)
- `Enum.ordinal()` returns the position (0-based) — fragile for persistence; use a dedicated field instead
- `Enum.name()` returns the constant's name as declared
- `Enum.valueOf(String)` performs a case-sensitive lookup

Thread safety: Enum singletons are thread-safe by the JVM's class loading guarantees — the JVM guarantees each enum constant is initialized exactly once.

Enum as singleton: The most reliable singleton implementation in Java, immune to reflection attacks (unlike regular singletons) because the JVM prohibits creating enum instances via reflection.

Real-Life Analogy

An Enum is like a **traffic light system** — there's a fixed, well-known set of states (RED, YELLOW, GREEN). Each state has specific behavior (RED means stop, GREEN means go) and properties (how long it lasts). You can't invent new traffic light colors at runtime, and you don't need to — the complete set is defined at design time.

Interview Context & Why It Matters

Advanced enum questions reveal whether you understand enums as full-fledged classes. The state pattern via enum, enum as a strategy pattern, and safe singleton via enum are senior-level concepts. Interviewers also test `EnumSet` and `EnumMap` which are $O(1)$ operations using bit manipulation.

Production-Ready Code Snippet

java

```java
import java.util.EnumMap;
import java.util.EnumSet;

/**
 * Demonstrates advanced enum usage: fields, state-specific methods,
 * abstract methods for per-constant behavior, interface implementation,
 * EnumSet/EnumMap, and the enum singleton pattern.
 */
public class AdvancedEnumDemo {

    // Enum with fields and methods — represents a Planet (classic Java example vari
    enum OrderStatus {
        PENDING("Awaiting payment", false) {
            @Override
            public OrderStatus nextStatus() { return PAID; }
        },
        PAID("Payment received", false) {
            @Override
            public OrderStatus nextStatus() { return PROCESSING; }
        },
        PROCESSING("Being fulfilled", false) {
            @Override
            public OrderStatus nextStatus() { return SHIPPED; }
        },
        SHIPPED("In transit", false) {
            @Override
            public OrderStatus nextStatus() { return DELIVERED; }
        },
        DELIVERED("Order complete", true) {
            @Override
            public OrderStatus nextStatus() {
                throw new IllegalStateException("DELIVERED is a terminal state");
            }
        },
        CANCELLED("Order cancelled", true) {
            @Override
            public OrderStatus nextStatus() {
                throw new IllegalStateException("CANCELLED is a terminal state");
            }
        };

        private final String description;
        private final boolean terminal;
```

```
    // Constructor: always private in enums
    OrderStatus(String description, boolean terminal) {
        this.description = description;
        this.terminal = terminal;
    }

    public String getDescription() { return description; }
    public boolean isTerminal() { return terminal; }

    // Abstract method: each constant MUST implement this
    public abstract OrderStatus nextStatus();
}

// Enum implementing an interface — strategy pattern
interface TaxCalculator {
    double calculateTax(double amount);
}

enum TaxRegion implements TaxCalculator {
    US_CALIFORNIA {
        @Override
        public double calculateTax(double amount) { return amount * 0.0725; }
    },
    UK {
        @Override
        public double calculateTax(double amount) { return amount * 0.20; }
    },
    GERMANY {
        @Override
        public double calculateTax(double amount) { return amount * 0.19; }
    },
    TAX_EXEMPT {
        @Override
        public double calculateTax(double amount) { return 0.0; }
    };

    // Shared utility method
    public double totalWithTax(double amount) {
        return amount + calculateTax(amount);
    }
}

// Enum Singleton — the most robust singleton implementation
```

```
enum AppConfig {
    INSTANCE;  // Single instance, thread-safe, reflection-proof

    private String databaseUrl = "jdbc:postgresql://localhost:5432/mydb";
    private int maxConnections = 10;

    public String getDatabaseUrl() { return databaseUrl; }
    public void setDatabaseUrl(String url) { this.databaseUrl = url; }
    public int getMaxConnections() { return maxConnections; }
}

public static void main(String[] args) {
    // State machine via enum
    OrderStatus status = OrderStatus.PENDING;
    System.out.println("Start: " + status + " — " + status.getDescription());

    while (!status.isTerminal()) {
        status = status.nextStatus();
        System.out.println("Transitioned to: " + status + " — " + status.getDesc
    }

    System.out.println();

    // Strategy pattern via enum
    double orderAmount = 100.0;
    for (TaxRegion region : TaxRegion.values()) {
        System.out.printf("%-15s total: $%.2f%n", region, region.totalWithTax(or
    }

    System.out.println();

    // EnumSet — highly efficient set backed by a long bitmask ($O(1)$ operation
    EnumSet<OrderStatus> activeStatuses = EnumSet.of(
        OrderStatus.PENDING, OrderStatus.PAID, OrderStatus.PROCESSING, OrderStat
    );
    EnumSet<OrderStatus> terminalStatuses = EnumSet.complementOf(activeStatuses)
    System.out.println("Terminal statuses: " + terminalStatuses);
    System.out.println("Is DELIVERED terminal? " + terminalStatuses.contains(Ord

    // EnumMap — hash map optimized for enum keys ($O(1)$, more efficient than H
    EnumMap<TaxRegion, String> regionNames = new EnumMap<>(TaxRegion.class);
    regionNames.put(TaxRegion.US_CALIFORNIA, "California, United States");
    regionNames.put(TaxRegion.UK, "United Kingdom");
    regionNames.put(TaxRegion.GERMANY, "Germany");
```

```
        System.out.println("Region names: " + regionNames);

        // Enum singleton
        AppConfig config = AppConfig.INSTANCE;
        System.out.println("DB URL: " + config.getDatabaseUrl());
        System.out.println("Singleton? " + (config == AppConfig.INSTANCE)); // true

        // Enum lookup
        OrderStatus paid = OrderStatus.valueOf("PAID");
        System.out.println("Looked up: " + paid.ordinal() + " → " + paid); // 1 → PA
    }
}
```

Common Interview Question & Expert Answer

Q: "Why is an enum-based singleton considered superior to a double-checked locking singleton? And what is the risk of using `ordinal()` for database persistence?"

A: "An enum singleton is superior because: (1) **Thread safety** is guaranteed by the JVM class loader without any synchronization code — the constants are initialized at class loading which is guaranteed to happen exactly once. (2) **Reflection safety** — `java.lang.reflect` throws an `IllegalArgumentException` when you try to call `newInstance()` on an enum class. A double-checked locking singleton can be broken by a sophisticated reflection attack that calls the private constructor. (3) **Serialization safety** — the JVM guarantees that deserializing an enum always returns the same constant; with a regular class, you must implement `readResolve()` to prevent deserialization from creating a second instance. Regarding `ordinal()` for persistence: **never use `ordinal()` for database storage**. The ordinal is the positional index (0, 1, 2...) and it changes if you reorder the enum constants or insert a new one in the middle. Existing database rows would now point to the wrong status. The correct approach is to use a dedicated `private final String code` or `private final int code` field in the enum and use that for persistence."